AI — Exam Practice Problems (Lectures 1–4, up to A*)

BS AI · Semester 3 · Concept-based, **no programming** · Every problem has a fully worked solution at the back (verified by running the algorithms in code).

How to use this: Cover the solutions section. Give yourself ~12–15 minutes per problem (the real paper has 3 problems, so a mock paper = 3 problems in about 45 minutes). Write the trace tables by hand exactly the way sir's slides do — frontier after each step, g/h/f columns, final path and cost.

Honest note about my guess of the paper: I only have the 4 lectures you uploaded, so these problems follow the style of those slides (problem formulation, BFS/DFS/UCS traces, Greedy and A* traces, admissibility/consistency, heuristics, agents/PEAS). Hill Climbing is in Lecture 4's slides, so I included it — but confirm with sir whether it counts as "till A*". The real paper may phrase things differently, so don't memorise these answers; practise the *reasoning*.

Conventions used in every search problem (state these on your paper too — it prevents arguments about tie-breaks):
1) Graph search with an **explored set**; a node already explored is never expanded again (unless the problem says re-opening is allowed).
2) **Goal test is done when a node is popped** (selected for expansion), not when it is generated — this is what makes UCS/A* optimal.
3) Ties are broken **alphabetically** (for A*: lowest f, then lowest h, then alphabetical). DFS pushes children so the alphabetically-first child is expanded first.
4) Duplicate entries for the same state may sit in the frontier (no decrease-key); a stale one is skipped when popped. Edges are directed as drawn.

PART A — Agents and Environments (Lectures 1–2)

Problem 1 — Classify the environment [8 marks]

For each task environment fill the six properties: **Observable** (Fully/Partially), **Deterministic/Stochastic**, **Episodic/Sequential**, **Static/Dynamic**, **Discrete/Continuous**, **Single/Multi-agent**. Use the lecture's convention that chess has no chance element.

(a) Chess with a clock (b) Poker (c) A crossword puzzle (d) A self-driving taxi in city traffic (e) A spam filter that classifies each incoming e-mail independently

(f) The lecture says "most real-world environments are stochastic, at least to some degree". Give **one** reason a robot arm in a perfectly rule-based factory can still be called stochastic.

(g) Someone claims: "Chess is multi-agent, therefore it is stochastic." Do you agree? Justify in two lines.

Problem 2 — Reflex vacuum agent and its limits [10 marks]

A vacuum world has two squares, A and B. The agent senses only **its location** and **whether that square is Dirty or Clean**. Actions: Suck, Right, Left, NoOp. Performance measure: +1 for every clean square at the end of each time step, −1 for every move (Left/Right).

(a) Write the four IF–THEN rules of a simple reflex agent for this world.

(b) Run it from (A, Dirty) with B also dirty for 6 time steps. Write the (location, status, action) sequence. What is wrong once both squares are clean?

(c) Can *any* simple reflex agent fix this problem? Explain precisely what information is missing. What is the **minimum** extra piece of internal state, and which agent type is that?

(d) A classmate says: "Just give the reflex agent a rule IF status = Clean THEN NoOp." Show a one-line counter-example.

Problem 3 — Agent types and rationality [8 marks]

(a) Choose the **weakest agent type that is sufficient** (simple reflex / model-based / goal-based / utility-based / learning) and justify in one sentence: (i) an automatic door, (ii) a GPS that returns *any* route to the destination, (iii) a GPS that trades off time, toll cost and accident risk, (iv) a robot vacuum that remembers which rooms it has cleaned, (v) a spam filter that improves from "mark as spam" clicks.

(b) True or False, with reasoning: (1) “A rational agent never makes a bad decision.” (2) “A self-driving car brakes for a shadow that looked like an obstacle. It was not rational.” (3) “More sophisticated agent types are always better.” (4) “Goal-based and utility-based agents are the same, since both reach the goal.”

Problem 4 — AI / ML / DL / GenAI map and PEAS [6 marks]

(a) Place each system in the **smallest correct layer** of AI ⊃ ML ⊃ DL ⊃ Generative AI (or say “AI but not ML”): (i) thermostat with hand-written rules, (ii) chess engine with a hand-coded evaluation function, (iii) spam filter trained on labelled e-mails (logistic regression), (iv) object detector in a self-driving car (CNN), (v) a chatbot that writes essays.

(b) Write a complete PEAS description of a **hospital medicine-delivery robot** (P, E, A, S — at least three items for P and E, two for A and S).

PART B — Problem Formulation and Uninformed Search (Lecture 3)

Problem 5 — Formulate the water-jug problem [10 marks]

You have a 3-litre jug and a 5-litre jug, both initially empty, and unlimited water. Goal: have **exactly 4 litres in the 5-litre jug**.

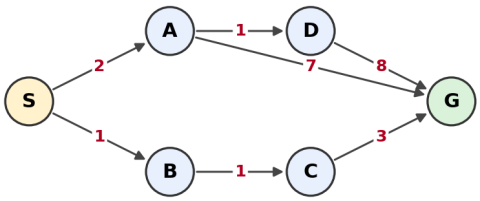
(a) Give the full formulation P = (S, A, Result, s_0, Goal, Cost): state representation, initial state, the actions A(s), Result(s, a) for the pour action, goal test, step cost.

(b) How many states are in the *full* set of combinations your representation allows? How many are actually **reachable** from s_0? Why do these differ?

(c) Find the shortest solution by BFS reasoning. How many actions? Is it unique in length?

(d) Is the “goal test” here a single goal state? Explain the lecture’s distinction between a goal *state* and a goal *test* using this problem.

Problem 6 — BFS vs DFS vs UCS trace [12 marks]



Start S, goal G. Edge costs as drawn (directed left→right). Use the conventions on page 1.

(a) Trace **BFS**, **DFS** and **UCS**: for each step write the node expanded, g(n), and the frontier afterwards. Give the final path, number of actions and total cost for each.

(b) Which algorithm(s) returned the optimal path? Which returned the path with the fewest *actions*?

(c) Explain why BFS returns a different answer from UCS here, using the lecture’s own sentence about what BFS optimises.

(d) Change **one** edge cost so that BFS’s path becomes optimal too. Which edge and to what value?

Problem 7 — Properties, complexity and “true or false” [10 marks]

(a) Branching factor b = 10, shallowest goal at depth d = 5, maximum depth m = 20. Using the lecture’s formulas: (i) how many nodes does BFS generate in levels 0…5? (ii) How many nodes does BFS hold in the frontier at depth 5? (iii) The DFS memory bound b·m? (iv) By what factor is BFS memory larger?

(b) UCS with b = 3, optimal cost $C^* = 10$, smallest step cost $\varepsilon = 2$. Evaluate the lecture’s bound $O(b^{1+\lfloor C^*/\varepsilon \rfloor})$. Why is it wrong to write it as $O(b^d)$?

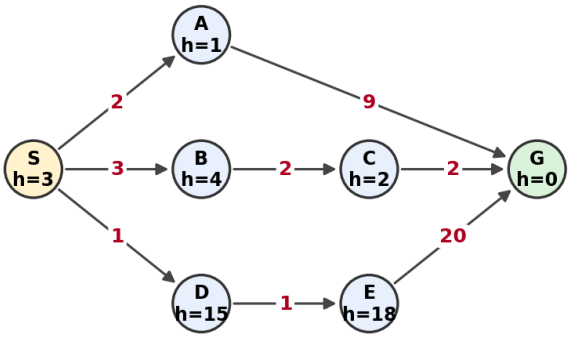
(c) True/False + one-line reason: (1) BFS finds the cheapest path whenever all costs are positive. (2) DFS with an explored set is complete on any *finite* graph. (3) UCS is guaranteed complete even if some edges cost 0. (4) If every step costs the same, UCS expands the same nodes as BFS (up to ties). (5) DFS is always faster than BFS. (6) “Path length” means the number of states on the path.

(d) Disagreement check: the lecture slide says “DFS is not complete — tracking an explored set fixes cycles but not an infinite branch.” Is DFS+explored set complete on a finite graph? On an infinite

graph with finite b? Explain the difference carefully.

PART C — Informed Search, Heuristics, A*, Local Search (Lecture 4)

Problem 8 — Greedy vs UCS vs A* [14 marks]



Start S, goal G. Heuristic values h(n) are written inside each node. Directed edges, costs as drawn.

(a) Compute the true remaining cost h*(n) for every node. Verify that h is admissible.

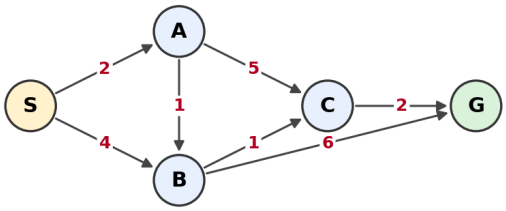
(b) Verify consistency on **all eight** edges (write each inequality).

(c) Trace Greedy Best-First (key h), UCS (key g) and A* (key f = g + h). For A* show a table of Expanded | g | h | f | Frontier after.

(d) Compare: path, cost, and number of nodes expanded for the three. Explain *why* Greedy is fooled and why A* is not.

(e) If h(n) = 0 for every node, which algorithm is A*? If h = h*, how many wasted expansions?

Problem 9 — Admissible? Consistent? And the lecture's final challenge [14 marks]



n	S	A	B	C	G
$h_1(n)$	5	4	3	3	0
$h_2(n)$	6	4	1	2	0
$h_3(n)$	6	4	3	2	0

(a) Compute h*(n). Classify each of h_1, h_2, h_3 as: admissible? consistent? Name every violating node/edge.

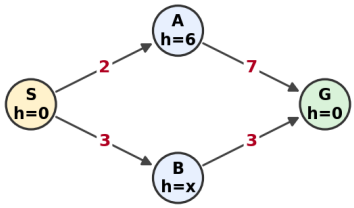
(b) Run **A* graph search with an explored set that never re-opens a node** using h_2. What path and cost does it return? Is it optimal?

(c) The lecture asks: "h is admissible but fails consistency on one edge — is graph-search A* still optimal? What must the code handle?" Answer using part (b), and show the fix on the same graph.

(d) h_1 is *not* admissible. Run A* with h_1. Does it return a suboptimal path? What does this tell you about the statement "inadmissible ⇒ A* is wrong"?

(e) Prove or disprove: consistent ⇒ admissible. Is the converse true? Which heuristic above is your witness?

Problem 10 — How wrong can h be? [8 marks]



Edges S→A 2, S→B 3, A→G 7, B→G 3. h(S)=0, h(A)=6, h(G)=0 and h(B)=x is unknown. Tie-break: lowest f, then lowest h.

(a) Find h*(n) and the optimal path. (b) For which x is h admissible? (c) For which x does A* return the **suboptimal** path S→A→G? Derive the threshold using f-values. (d) Is there a range of x where h *overestimates* but A* is still optimal? Explain what that means for the "overestimation trap" slide.

Problem 11 — 8-puzzle heuristics [12 marks]

Goal state (as in the lecture): **1 2 3 / 8 _ 4 / 7 6 5**. Start state n: **2 8 1 / 4 _ 3 / 7 6 5** (rows separated by "/", blank = _). Every move costs 1.

(a) Compute h_1 (misplaced tiles, blank excluded) and h_2 (Manhattan) with a per-tile table.

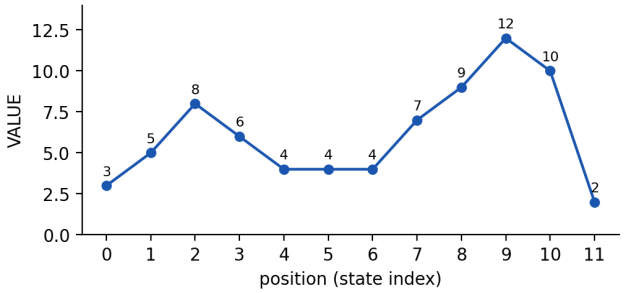
(b) The true optimal solution length is h*(n) = 10. Check $h_1 \le h_2 \le h^*$. Which dominates and what does dominance guarantee — and what does it **not** guarantee?

(c) A student proposes $h_3 = h_1 + h_2$. Compute $h_3(n)$. Is h_3 admissible? Give a general counter-example from a state one move away from the goal.

(d) A student counts the blank as a tile in h_1. Show why that breaks admissibility.

(e) Prove that Manhattan distance is consistent for the 8-puzzle with unit step cost.

Problem 12 — Hill climbing: maxima, plateaus, and 4-queens [12 marks]



One-dimensional landscape; neighbours of position i are i−1 and i+1. Steepest-ascent hill climbing as in the lecture: move to the best neighbour; stop when VALUE(neighbour) ≤ VALUE(current).

(a) Where does it stop when started at positions 0, 4, 5, 6, 11? Label each stop as global maximum, local maximum or plateau stop. (b) If start positions are chosen uniformly at random, what is the probability of reaching the global maximum? Does restarting help, and how many restarts do you expect on average?

(c) 4-queens, one queen per column, state = row of the queen in each column, h = number of attacking pairs. State (1, 3, 4, 2): compute h and list the attacking pair(s). Compute h for all 12 neighbours (change one queen's row). Does the lecture's algorithm stop here although h ≠ 0? Solutions exist — what kind of move must the search accept to reach one?

Problem 13 — Choose the strategy [6 marks]

Pick the most suitable algorithm (BFS / DFS / UCS / Greedy / A* / Hill Climbing) and justify in one line: (i) unweighted maze, need the fewest moves; (ii) very deep search tree, tiny memory, any solution is acceptable; (iii) road map with distances but no coordinates, need the cheapest route; (iv) road map with GPS coordinates, need the cheapest route; (v) 1000-queens, only the final board matters; (vi) fast approximate route, a reliable h exists, optimality not required.

PART D — Rapid-fire conceptual questions (answer in 1–2 lines, check below)

R1. What is the difference between a goal *state* and a goal *test*?

R2. Why do UCS and A* test for the goal when a node is *popped*, not generated?

R3. Which algorithm is A* when h(n) = 0 for all n?

R4. What is the difference between h(n) and h*(n)? Which one does the algorithm compute?

R5. Can a heuristic be consistent but not admissible? (h(goal) = 0.)

R6. Which condition is sufficient for A* *tree* search to be optimal? For *graph* search?

R7. State vs node vs search tree vs state-space graph: one line each.

R8. What does an explored set protect against, and what does it *not* protect DFS against?

R9. Rational vs omniscient — which one can still lose?

R10. Goal-based vs utility-based: what does the second add?

R11. Does heuristic dominance mean A* with h_2 expands *strictly fewer* nodes?

R12. Local search vs systematic search: what do you gain and what do you lose?

Suggested mock papers (45 min each, pick 3 problems):
Mock 1: Problem 6 + Problem 8 + Problem 9 · **Mock 2:** Problem 5 + Problem 10 + Problem 11 · **Mock 3:** Problem 2 + Problem 7 + Problem 12
Since sir said the paper rewards *understanding* over rote learning, expect "why / what happens if / is this statement true?" parts attached to the traces.

SOLUTIONS

Part A — Agents and Environments

Solution 1

Environment	Observable	Det./Stoch.	Epis./Seq.	Static/Dyn.	Disc./Cont.	Single/Multi
(a) Chess + clock	Fully	Deterministic	Sequential	**Semidynamic** (clock changes score, not the board)	Discrete	Multi
(b) Poker	**Partially** (hidden cards)	Stochastic (shuffle)	Sequential	Static	Discrete	Multi
(c) Crossword	Fully	Deterministic	Sequential	Static	Discrete	Single
(d) Self-driving taxi	Partially	Stochastic	Sequential	Dynamic	Continuous	Multi
(e) Spam filter (per e-mail)	Partially (intent, context hidden)	Stochastic	**Episodic**	Static per decision	Discrete (spam/ham)	Single

(f) Actuator noise / wheel slip / sensor error: the same command can produce a slightly different position, so the transition is not perfectly predictable. **(g) Disagree.** Multi-agent and stochastic are independent axes. Chess is multi-agent but has no chance element (deterministic in the lecture's convention); poker is both. Multi-agent means other agents' actions affect your performance — it says nothing about randomness. (Strict textbooks call chess "strategic", but the point stands.)

Solution 2

(a) IF location = A AND status = Dirty THEN Suck · IF location = B AND status = Dirty THEN Suck · IF location = A AND status = Clean THEN Right · IF location = B AND status = Clean THEN Left.

(b) t1 (A,Dirty)→Suck · t2 (A,Clean)→Right · t3 (B,Dirty)→Suck · t4 (B,Clean)→Left · t5 (A,Clean)→Right · t6 (B,Clean)→Left … After t3 both squares are clean, yet the agent keeps shuttling forever, paying −1 per move. It is **not rational**: a better action (NoOp) exists.

(c) No simple reflex agent can fix it: the percept (A, Clean) is *identical* whether B is dirty or already clean, and a simple reflex agent maps the **current percept only** to an action, so it must give the same action in both cases. The missing information is **whether the other square is clean**. The minimum fix is a small internal state: a clean-flag per square (2 bits: "A known clean", "B known clean"). That makes it a **model-based reflex agent**: IF both flags = clean THEN NoOp.

(d) Start (A, Clean) with B dirty: the rule says NoOp, so B is never cleaned and the agent stays at the initial dirt forever — the opposite failure. Rules that depend only on the current percept cannot be right for both situations.

Solution 3

(a) (i) Simple reflex — depends only on "person detected now". (ii) Goal-based — searches alternative routes to reach the destination. (iii) Utility-based — several routes satisfy the goal; utility ranks them by time/cost/risk. (iv) Model-based reflex — needs memory of cleaned rooms (it may also be goal-based if it plans a coverage route). (v) Learning — improves from feedback (critic + learning element).

(b) (1) **False.** Rational = best *expected* action given percepts and knowledge; outcomes can still be bad (not omniscient). (2) **False claim** — braking for a possible obstacle with the available percept can be rational; rationality is judged at decision time, not in hindsight. (3) **False** — the right type depends on the task; a learning agent for a thermostat is wasted cost and complexity. (4) **False** — a goal is binary (reached or not); utility is a matter of degree and chooses the *best* way among several goal-reaching routes.

Solution 4

(a) (i) AI but not ML (hand-written rules). (ii) AI but not ML (hand-coded evaluation + search). (iii) ML (not deep learning). (iv) Deep learning (a CNN; it is ML as well). (v) Generative AI (built on deep learning / foundation models, commonly transformers).

	Hospital medicine-delivery robot
P	Delivery accuracy (right medicine, right patient), safety of people, delivery time, battery use, no collisions
E	Hospital corridors, lifts, doors, patients and staff, trolleys, nurse stations, wards
A	Wheels/drive motors, lift-button arm, drawer/locker lock, speaker/screen, alarm
S	Cameras, LiDAR/ultrasonic distance sensors, bump sensors, RFID/barcode scanner, map/Wi-Fi location

Part B — Problem formulation and uninformed search

Solution 5

(a) State (a, b): a = litres in the 3-L jug (0–3), b = litres in the 5-L jug (0–5). s_0 = (0, 0). **Actions A(s)**: Fill3, Fill5, Empty3, Empty5, Pour3→5, Pour5→3 (only those that change the state are meaningful). **Result**: Fill3 → (3, b); Fill5 → (a, 5); Empty3 → (0, b); Empty5 → (a, 0); Pour3→5: t = min(a, 5−b), result (a−t, b+t); Pour5→3: t = min(b, 3−a), result (a+t, b−t). **Goal(s)**: b = 4. **Cost**: 1 per action (so path cost = number of actions).

(b) The representation allows 4 × 6 = **24** tuples, but only **16 are reachable** from (0,0). The 8 unreachable ones are (1,1)…(1,4) and (2,1)…(2,4): every action ends with some jug being filled, emptied, or poured until the source is empty or the target is full, so in every reachable state **at least one jug is empty or full** (a ∈ {0,3} or b ∈ {0,5}). The state space is the set of states *reachable* from s_0, not every tuple the notation can write.

(c) BFS level by level: (0,0) → (0,5) [Fill5] → (3,2) [Pour5→3] → (0,2) [Empty3] → (2,0) [Pour5→3] → (2,5) [Fill5] → (3,4) [Pour5→3]. **6 actions**, ending in (3,4). It is the **only** shortest solution (I counted shortest paths by search); the other reachable goal state (0,4) needs 7 actions (e.g. the same 6 plus Empty3). BFS guarantees the fewest actions because every action costs 1.

(d) Goal(s) is a *test*, "b = 4", satisfied by four tuples: (0,4), (1,4), (2,4), (3,4). So the problem has several goal *states*, and the test handles them uniformly. A goal state is one particular state that satisfies the objective. (Only (3,4) and (0,4) are reachable — a nice illustration that a goal test can accept states the search will never see.)

Solution 6

BFS (FIFO):

Step	**Expanded**	**g**	**Frontier after (front → back)**
1	S	0	A(2), B(1)
2	A	2	B(1), D(3), G(9)
3	B	1	D(3), G(9), C(2)
4	D	3	G(9), C(2), G(11)
5	pop G → **GOAL**	9	—

Path **S→A→G**, 2 actions, cost **9**. (g shown in brackets for reference; BFS ignores it.)

DFS (LIFO, top of stack on the right):

Step	**Expanded**	**Stack after (bottom → top)**
1	S	B, A
2	A	B, G(9), D
3	D	B, G(9), G(11)
4	pop G(11) → **GOAL**	B, G(9)

Path **S→A→D→G**, 3 actions, cost **11** — the worst of the three; it dives A→D→G and stops at the first goal.

UCS (priority queue on g):

Step	**Expanded**	**g**	**Frontier after {node:g}**
1	S	0	A:2, B:1

Step	Expanded	g	Frontier after {node:g}
2	B	1	A:2, C:2
3	A (tie with C, alphabetical)	2	C:2, D:3, G:9
4	C	2	D:3, G:9, G:5
5	D	3	G:9, G:5, G:11
6	pop G:5 → **GOAL**	5	stale G:9, G:11 discarded

Path **S→B→C→G**, 3 actions, cost **5**.

(b) Only **UCS** is optimal (cost 5). Only **BFS** has the fewest actions (2). **(c)** BFS optimises the *number of actions*, not cost; it stops at the shallowest goal and never notices that a 3-edge path (5) beats the 2-edge path (9). **(d)** Lower c(A,G) from 7 to 3 (or less). Then S→A→G costs 2 + 3 = 5, which ties with S→B→C→G (5), so BFS's 2-edge path is optimal too; any c(A,G) ≤ 3 works.

Solution 7

(a) (i) $1 + 10 + 10^2 + 10^3 + 10^4 + 10^5$ = **111,111** nodes. (ii) The frontier at depth 5 holds about b^d = **100,000** nodes. (iii) b·m = 10 × 20 = **200** nodes. (iv) 100,000 / 200 = **500×** more memory for BFS. (Exact counts depend on when the goal test is applied; the lecture uses $O(b^d)$.)

(b) $3^{1+\lfloor 10/2 \rfloor} = 3^6$ = **729**. UCS orders by cost, not depth, so its work is governed by how many steps of size ε fit under C* — a shallow goal with a huge cost can make UCS explore far more than b^d; with tiny ε the bound explodes even if d is small.

(c) (1) **False** — BFS minimises actions; needs uniform cost. (2) **True** — finite graph + explored set ⇒ every state is expanded at most once, so it terminates, and it finds a goal if one is reachable (though not necessarily the best). (3) **Not guaranteed** — completeness needs step costs ≥ ε > 0; with a zero-cost cycle and no explored set UCS can loop forever. (4) **True** — g(n) = depth(n), so the priority order equals the FIFO order up to ties. (5) **False** — time is $O(b^m)$, and m can be far larger than d. (6) **False** — path length is the number of **actions** (states − 1).

(d) The lecture statement is correct but easy to over-read. On a **finite** graph, DFS + explored set **is** complete (state space is finite, nothing is expanded twice). On an **infinite** (or effectively infinite) graph, the explored set does not help: DFS can commit to an infinite acyclic branch without ever repeating a state, and never backtracks to the goal on another branch. So: explored set cures *cycles*, not *infinite depth*.

Part C — Informed search

Solution 8

(a) h*: G = 0, C = 2, E = 20, A = 9, B = 2+2 = **4**, D = 1+20 = **21**, S = min(2+9, 3+4, 1+21) = **7**. Compare with h: S 3≤7, A 1≤9, B 4≤4, C 2≤2, D 15≤21, E 18≤20, G 0≤0 → **admissible**.

(b) h(u) ≤ c(u,v) + h(v): S→A 3 ≤ 2+1 = 3 ✓ · S→B 3 ≤ 3+4 = 7 ✓ · S→D 3 ≤ 1+15 = 16 ✓ · A→G 1 ≤ 9+0 ✓ · B→C 4 ≤ 2+2 = 4 ✓ · C→G 2 ≤ 2+0 = 2 ✓ · D→E 15 ≤ 1+18 = 19 ✓ · E→G 18 ≤ 20+0 ✓. All hold ⇒ **consistent** (so f never decreases along any path).

(c) Greedy (key h): S → frontier A(h=1), B(4), D(15) → pop A (lowest h) → frontier B, D, G(h=0) → pop G. Expansions: S, A (then G popped). Path **S→A→G, cost 11**.

UCS: S(0) → D(1) → A(2) / E(2) tie → … Order of expansion: **S, D, A, E, B, C**, then G(7) is popped. Path S→B→C→G, cost 7 (6 expansions).

A*:

Step	Expanded	g	h	f	Frontier after {node: f}
1	S	0	3	3	A:3, B:7, D:16
2	A	2	1	3	B:7, G:11, D:16
3	B	3	4	7	C:7, G:11, D:16
4	C	5	2	7	G:7 (improves 11→7), D:16
5	pop G → **GOAL**	7	0	7	D:16 never expanded

(d) Greedy: cost 11 (not optimal), 2 expansions. UCS: cost 7, 6 expansions. A*: cost 7, 4 expansions (S, A, B, C) before popping G. Greedy ignores g, so it jumps to A because h(A) = 1 *looks* closest, never accounting for the 9 it still has to pay. A* adds g, so A's f = 3 only gets it a quick look; the 11-cost goal

entry loses to C's f = 7, and D (f = 16) is pruned by its large h. **(e)** h ≡ 0 ⇒ f = g ⇒ A* **is UCS**. h = h* ⇒ A* expands only nodes on an optimal path (zero wasted expansions).

Solution 9

(a) h*: S = 6 (S→A→B→C→G = 2+1+1+2), A = 4, B = 3, C = 2, G = 0.

	Admissible?	Consistent?	Violations
h_1	**No**	**No**	$h_1(C) = 3 > h^*(C) = 2$; edge C→G: 3 ≤ 2+0 fails
h_2	Yes (6≤6, 4≤4, 1≤3, 2≤2)	**No**	S→B: 6 ≤ 4+1 = 5 fails; A→B: 4 ≤ 1+1 = 2 fails. All other edges hold
h_3	Yes (= h*)	Yes	none — h* itself is always consistent

(b) A* with h_2 (explored set, no re-opening):

Step	Expanded	g	h	f	Frontier after {node: f}
1	S	0	6	6	A:6 (g2), B:5 (g4)
2	B	4	1	5	A:6, C:7 (g5), G:10 (g10)
3	A	2	4	6	C:7, G:10; B reached again with g = 3 but B is already explored → ignored; C via A has g = 7 → not better
4	C	5	2	7	G:7 (g7)
5	pop G → GOAL	7	0	7	—

Returned: **S→B→C→G, cost 7**. Optimal is S→A→B→C→G = **6**. **Not optimal.** Notice f dropped from f(S) = 6 to f(B) = 5 — f decreasing along a path is the symptom of inconsistency.

(c) No — with an admissible but inconsistent h, graph-search A* is **not** guaranteed optimal if an explored node is never revisited: B was closed with g = 4, but a cheaper g = 3 was found later via A. **The code must re-open:** if g_new < g(child), update the child and put it back into the frontier *even if it is in the explored set* (the lecture's pseudocode line "if unseen or g_new < g(child): update g,f; requeue" does exactly this, but a version with a strict explored check would not). Fix on the graph: when A is expanded, B gets g = 3, f = 3+1 = 4 and returns to the frontier; B is re-expanded: C improves to g = 4 (f = 6), G to g = 9; then C is expanded: G improves to g = 6 → goal S→A→B→C→G, cost 6. Alternative fix: use a consistent heuristic (h_3).

(d) Run with h_1: expand S (f=5); A f = 2+4 = 6, B f = 4+3 = 7 → expand A; B improves to g = 3, f = 6; C g = 7 f = 10 → expand B; C g = 4, f = 7; G g = 9 f = 9 → expand C; G g = 6, f = 6 → pop G. Result: S→A→B→C→G, cost **6 — optimal**. So an inadmissible h **does not automatically** make A* wrong; it only removes the guarantee. Overestimating C (3 instead of 2) was too small to hide the optimal path. (Disagree with the shortcut "overestimate ⇒ wrong answer": it is "overestimate *enough* ⇒ can be wrong".) See Problem 10.

(e) True. Proof: induct on the number of steps k to the goal along an optimal path. k = 0: h(goal) = 0 = h*(goal). Step: for the first edge n→n′ of an optimal path, $h(n) \le c(n,n') + h(n') \le c(n,n') + h^*(n') = h^*(n)$. Converse **false**: h_2 is admissible but not consistent.

Solution 10

(a) h*(B) = 3, h*(A) = 7, h*(S) = 6. Optimal path **S→B→G, cost 6**. **(b)** Admissible iff 0 ≤ x ≤ 3 (A: 6 ≤ 7 ✓ already).

(c) After expanding S: f(A) = 2 + 6 = 8 and f(B) = 3 + x. **Case x ≤ 5:** f(B) ≤ 8 and (at the tie x = 5, f = 8) B has the lower h, so B is expanded first; G via B gets f = 6 and is popped before anything else → optimal S→B→G (cost 6). **Case x ≥ 6:** f(B) ≥ 9 > 8, so A is expanded first and puts G (g = 9, f = 9) into the frontier. Now B (f = 3 + x ≥ 9) competes with G (f = 9): for x > 6 G wins outright; at x = 6 the tie f = 9 goes to the lower h, i.e. G. So **A* returns the suboptimal S→A→G (cost 9) exactly when x ≥ 6** (strictly x > 6 regardless of tie-breaking). Computed check: x = 0…5 → cost 6; x = 6…9 → cost 9.

(d) Yes: **3 < x < 6** overestimates h*(B) = 3 but A* is still optimal, because B's inflated f = 3 + x stays below the cost 9 of the wrong goal. The "overestimation trap" slide shows h(B) = 8, a large overestimate; the correct statement is: overestimation **can** hide the optimal path, and it does exactly when f(optimal path) exceeds the cost of some suboptimal goal. Admissibility is a *sufficient* condition for optimality, not a necessary one.

Solution 11

Tile	Current (row,col)	Goal (row,col)	Manhattan
1	(1,3)	(1,1)	0+2 = 2
2	(1,1)	(1,2)	0+1 = 1
3	(2,3)	(1,3)	1+0 = 1
4	(2,1)	(2,3)	0+2 = 2
5	(3,3)	(3,3)	0
6	(3,2)	(3,2)	0
7	(3,1)	(3,1)	0
8	(1,2)	(2,1)	1+1 = 2
		h_2 total	**8**

(a) Misplaced tiles: 1, 2, 3, 4, 8 → **$h_1 = 5$** (5, 6, 7 are in place). **$h_2 = 8$**.

(b) $5 \le 8 \le 10$ ✓. **h_2 dominates h_1** (both admissible, $h_2 \ge h_1$ everywhere): A* with h_2 **never expands more** nodes than with h_1. It does *not* promise strictly fewer — the lecture's own example (slide 37) has equal counts of 5 — and h_2 costs more per node to compute.

(c) $h_3 = 5 + 8 =$ **$13 > h^* = 10$**, so **not admissible**. General counter-example: a state one move from the goal has $h_1 = 1$, $h_2 = 1$, $h^* = 1$ but $h_3 = 2 > 1$. Sum of two admissible heuristics counts the same unavoidable move twice. The safe combination is **$\max(h_1, h_2)$** (here = h_2).

(d) If the blank is counted, a state one move from the goal has two "misplaced" items (the tile and the blank), so $h_1 = 2 > h^* = 1$ → overestimates. Hence "the blank is never counted."

(e) A move slides exactly one tile exactly one square, so one tile's Manhattan term changes by exactly ±1 and every other term is unchanged. So $h_2(n) - h_2(n') \le 1 = c(n,n')$ for every edge, i.e. $h_2(n) \le c(n,n') + h_2(n')$. With $h_2(\text{goal}) = 0$ this gives consistency (and hence admissibility). The same argument works for h_1 (a move changes the misplaced count by at most 1).

Solution 12

(a) Values: 0→3, 1→5, 2→8, 3→6, 4→4, 5→4, 6→4, 7→7, 8→9, 9→12, 10→10, 11→2.

Start	Path	Stops at	Type
0	0→1→2	2 (value 8)	local maximum
4	4→3→2	2 (value 8)	local maximum
5	—	5 (value 4)	**plateau** stop: both neighbours = 4, not strictly better
6	6→7→8→9	9 (value 12)	**global maximum**
11	11→10→9	9 (value 12)	global maximum

(b) Starts 6–11 succeed (6 of 12), starts 0–5 fail → **P = 6/12 = 50%**. Random restarts help: each run succeeds with $p = 0.5$, so the expected number of runs is $1/p =$ **2**; the chance that all of k runs fail is 0.5^k, e.g. 3 restarts leave a 12.5% failure chance. (Restarts keep O(1) memory — that is why the lecture lists them as a "fix for later".)

(c) State (1,3,4,2): queens at (col1,row1), (col2,row3), (col3,row4), (col4,row2). Only **Q2–Q3** attack (adjacent columns, adjacent rows → same diagonal) ⇒ **h = 1**. Neighbour h-values (row to which the queen moves):

Column	row 1	row 2	row 3	row 4
Q1	current	4	2	3
Q2	**1**	2	current	2
Q3	2	2	3	current
Q4	3	current	3	3

Best neighbour has h = 1 (Q2 → row 1, a **sideways** move), none has $h < 1$, so strict steepest descent (the lecture's "stop if not better") halts at a **local minimum with $h = 1 \neq 0$**. The solutions are (2,4,1,3) and (3,1,4,2). (3,1,4,2) differs from the current state in only 2 columns (Q1 → row 3, Q2 → row 1), but neither move alone improves h. To get there the search must accept a **non-improving move**: sideways Q2 → row 1 gives (1,1,4,2) with h = 1, and then Q1 → row 3 gives (3,1,4,2) with h = 0. So **limited sideways moves** solve this instance; random restarts or simulated annealing (accept some

worse moves) are the general fixes. The lecture's plain algorithm does *not* take sideways moves, which is exactly why it gets stuck.

Solution 13

(i) **BFS** — unweighted, fewest actions, memory affordable. (ii) **DFS** — O(b·m) memory, no optimality needed. (iii) **UCS** — weighted, no heuristic available. (iv) **A*** with straight-line distance (admissible and consistent). (v) **Hill climbing** — path irrelevant, huge space, O(1) memory (with restarts). (vi) **Greedy best-first** — fast when h is reliable, but with no optimality guarantee.

Part D — Rapid-fire answers

R1. Goal state = one particular state that satisfies the objective; goal test = the procedure Goal(s) that checks *any* state (can accept many goal states, e.g. any checkmate).

R2. A goal reached through an expensive path can be generated early; only when it is popped (lowest g or f in the frontier) is it certain that no cheaper path remains.

R3. UCS (f = g).

R4. h(n) = the algorithm's estimate (cheap, computed); h*(n) = the true optimal cost to go, unknown, used only in proofs and admissibility checks.

R5. No. Consistency + h(goal) = 0 implies h ≤ h* (admissible), so consistent ⇒ admissible; the reverse fails.

R6. Tree search: admissible h. Graph search (explored set, no re-opening): consistent h (or re-open nodes when a cheaper g appears).

R7. State = configuration of the world, no history. Node = bookkeeping record (state, parent, action, g, depth). Search tree = nodes the algorithm actually builds (same state can appear many times). State-space graph = full problem graph, independent of the algorithm.

R8. It prevents re-expanding states and so protects against *cycles*; it does not protect DFS against an infinite or extremely deep acyclic branch.

R9. The rational agent — it maximises expected performance with available information; omniscience is impossible, and a rational choice can still turn out badly.

R10. Utility ranks *how good* each way of reaching the goal is (speed, cost, safety); a goal-based agent only checks whether the goal is reached.

R11. No — it means "never more". In the lecture's example both h_1 and h_2 need 5 expansions; h_2 is simply never worse.

R12. Gain: constant memory, works on huge spaces, fine when only the final state matters. Lose: no path returned, no completeness/optimality guarantee, can stick at local maxima, plateaus and ridges.

Where students usually lose marks (all appear above): (1) Writing path length as number of states. (2) Saying BFS is optimal without "if step costs are equal". (3) Testing the goal at generation in UCS/A*. (4) Forgetting that UCS can update a frontier entry when a cheaper path appears. (5) Saying "inadmissible ⇒ always wrong" or "consistent and admissible are the same". (6) Treating $h_1 + h_2$ as admissible. (7) Confusing *complete* with *optimal*. (8) Not stating tie-break rules in a trace.